

Datos del Estudiante			
Carrera:	Asignatura:	Integrantes	
Desarrollo de Software	DevOops	Victor Hualpa John Sotomayor	
Periodo Lectivo:	Curso:	Fecha:	Docente:
2026-I	5to" A"	04-06-2026	Ing.Byron Moreno

Desarrollado por: Estudiante Victor Hualpa (Conceptos 1 al 7)

1. Control de versiones

- **Definición:** Sistema que registra los cambios en un archivo o conjunto de archivos a lo largo del tiempo.
- **Utilidad:** Permite a un equipo colaborar, rastrear el historial de modificaciones y recuperar versiones anteriores si ocurre un error.
- **Ejemplo práctico:** Revertir el proyecto a la versión del día anterior tras borrar por accidente una parte importante del código.
- **Fuente bibliográfica:** Chacon, S., & Straub, B. (2015). *Pro Git: El libro oficial en español* (2.^a ed.). Apress.

2. Git

- **Definición:** Sistema de control de versiones distribuido, de código abierto, muy rápido y gratuito.
- **Utilidad:** Permite a cada desarrollador tener una copia completa del proyecto, facilitando el trabajo sin conexión y la gestión eficiente de múltiples ramas.
- **Ejemplo práctico:** Ejecutar el comando `git init` en una carpeta de la computadora para empezar a registrar sus cambios.
- **Fuente bibliográfica:** Chacon, S., & Straub, B. (2015). *Pro Git: El libro oficial en español* (2.^a ed.). Apress.

3. GitHub

- **Definición:** Plataforma en la nube que aloja repositorios Git y facilita el trabajo colaborativo.
- **Utilidad:** Centraliza el código en internet, permite la revisión de proyectos en equipo y automatiza flujos de desarrollo.
- **Ejemplo práctico:** Subir el código a github.com para que tu compañero de grupo lo pueda descargar en su computadora.
- **Fuente bibliográfica:** GitHub Docs. (s.f.). *Acerca de GitHub*. Recuperado el 4 de junio de 2026, de <https://docs.github.com/es>

4. Repositorio local



- **Definición:** Carpeta en la computadora del usuario donde Git rastrea y almacena el historial del proyecto.

- **Utilidad:** Permite programar, probar ideas y guardar cambios localmente sin necesidad de internet.
- **Ejemplo práctico:** La carpeta "Trabajo_Git" en tu escritorio donde has modificado archivos y hecho tus commits.
- **Fuente bibliográfica:** Atlassian. (s.f.). *Tutoriales de Git: Repositorios*. Recuperado el 4 de junio de 2026, de <https://www.atlassian.com/es/git/tutorials>

5. Repositorio remoto

- **Definición:** Versión del proyecto alojada en un servidor de internet o red externa.
- **Utilidad:** Sirve como copia de seguridad en la nube y punto central para que el equipo sincronice su trabajo.
- **Ejemplo práctico:** Un repositorio público en GitHub al que los estudiantes se conectan mediante `git clone`.
- **Fuente bibliográfica:** Atlassian. (s.f.). *Tutoriales de Git: Sincronización*. Recuperado el 4 de junio de 2026, de <https://www.atlassian.com/es/git/tutorials>

6. Commit

- **Definición:** Captura permanente del estado de los archivos en un momento específico del historial.
- **Utilidad:** Guarda el progreso realizado y explica (mediante un mensaje) qué se modificó y por qué.
- **Ejemplo práctico:** Guardar un avance ejecutando `git commit -m "Se añade el título al documento"`.
- **Fuente bibliográfica:** Chacon, S., & Straub, B. (2015). *Pro Git: El libro oficial en español* (2.^a ed.). Apress.

7. Branch (rama)

- **Definición:** Línea de desarrollo independiente separada de la versión principal del código (rama `main`).
- **Utilidad:** Permite probar nuevas funciones o arreglar errores sin riesgo de dañar el proyecto base.
- **Ejemplo práctico:** Crear la rama `feature-texto` para escribir información nueva mientras el proyecto original sigue intacto.
- **Fuente bibliográfica:** Atlassian. (s.f.). *Uso de ramas en Git*. Recuperado el 4 de junio de 2026, de <https://www.atlassian.com/es/git/tutorials/using-branches>

Desarrollado por: Estudiante John Sotomayor (Conceptos 8 al 15)

8. Merge

- **Definición:** Proceso de unir el historial y los cambios de una rama secundaria hacia otra rama (generalmente la principal).



- **Utilidad:** Integra de forma definitiva el trabajo finalizado por un desarrollador al código base del equipo.

- **Ejemplo práctico:** Tras terminar un texto en la rama `feature-texto`, usar `git merge` para unirlo a la rama `main`.
- **Fuente bibliográfica:** Chacon, S., & Straub, B. (2015). *Pro Git: El libro oficial en español* (2.^a ed.). Apress.

9. Pull Request

- **Definición:** Petición formal en plataformas como GitHub para que otros aprueben tus cambios antes de unirlos al código principal.
- **Utilidad:** Fomenta la revisión de código entre compañeros, evitando que se integren errores al proyecto.
- **Ejemplo práctico:** Pedir por GitHub que tu compañero revise tu código; él lee los cambios y presiona "Approve".
- **Fuente bibliográfica:** GitHub Docs. (s.f.). *Acerca de las solicitudes de incorporación de cambios*. Recuperado el 4 de junio de 2026, de <https://docs.github.com/es>

10. Fork

- **Definición:** Copia exacta del repositorio de otro usuario que se guarda y aloja en tu propia cuenta.
- **Utilidad:** Permite proponer mejoras en proyectos públicos o de terceros sin tener permisos de administrador en el original.
- **Ejemplo práctico:** Copiar el proyecto de una herramienta pública a tu cuenta para probar un cambio por tu cuenta.
- **Fuente bibliográfica:** GitHub Docs. (s.f.). *Bifurcar un repositorio*. Recuperado el 4 de junio de 2026, de <https://docs.github.com/es>

11. Conflictos de integración

- **Definición:** Problema que ocurre cuando dos personas editan la misma línea de un archivo y Git no sabe cuál versión conservar.
- **Utilidad:** Evita sobrescrituras automáticas accidentales, obligando a los usuarios a decidir manualmente qué código es el correcto.
- **Ejemplo práctico:** Dos estudiantes cambian el título del proyecto a la vez; GitHub avisa del conflicto y deben borrar la línea incorrecta.
- **Fuente bibliográfica:** Atlassian. (s.f.). *Resolución de conflictos de Git*. Recuperado el 4 de junio de 2026, de <https://www.atlassian.com/es/git/tutorials/using-branches/merge-conflicts>

12. Integración Continua (CI)

- **Definición:** Práctica en la que el equipo integra su código frecuentemente, activando pruebas automáticas en un servidor.
- **Utilidad:** Detecta errores y problemas de compatibilidad de manera casi instantánea, mejorando la calidad del código.
- **Ejemplo práctico:** Cada vez que haces un `push`, un sistema revisa automáticamente si la aplicación sigue funcionando.



Fuente bibliográfica: Red Hat. (s.f.). *¿Qué es CI/CD?*. Recuperado el 4 de junio de 2026, de <https://www.redhat.com/es/topics/devops/what-is-ci-cd>

13. Entrega Continua (Continuous Delivery)

- **Definición:** Práctica que asegura que el código validado por la fase de CI siempre esté en un estado listo para enviarse a los clientes.
- **Utilidad:** Reduce el esfuerzo de lanzar actualizaciones, ya que el equipo puede publicarlas manualmente con un solo clic.
- **Ejemplo práctico:** Tras pasar las pruebas, un administrador presiona un botón en una web para actualizar la aplicación de los usuarios.
- **Fuente bibliográfica:** Amazon Web Services. (s.f.). *¿Qué es la entrega continua?*. Recuperado el 4 de junio de 2026, de <https://aws.amazon.com/es/devops/continuous-delivery/>

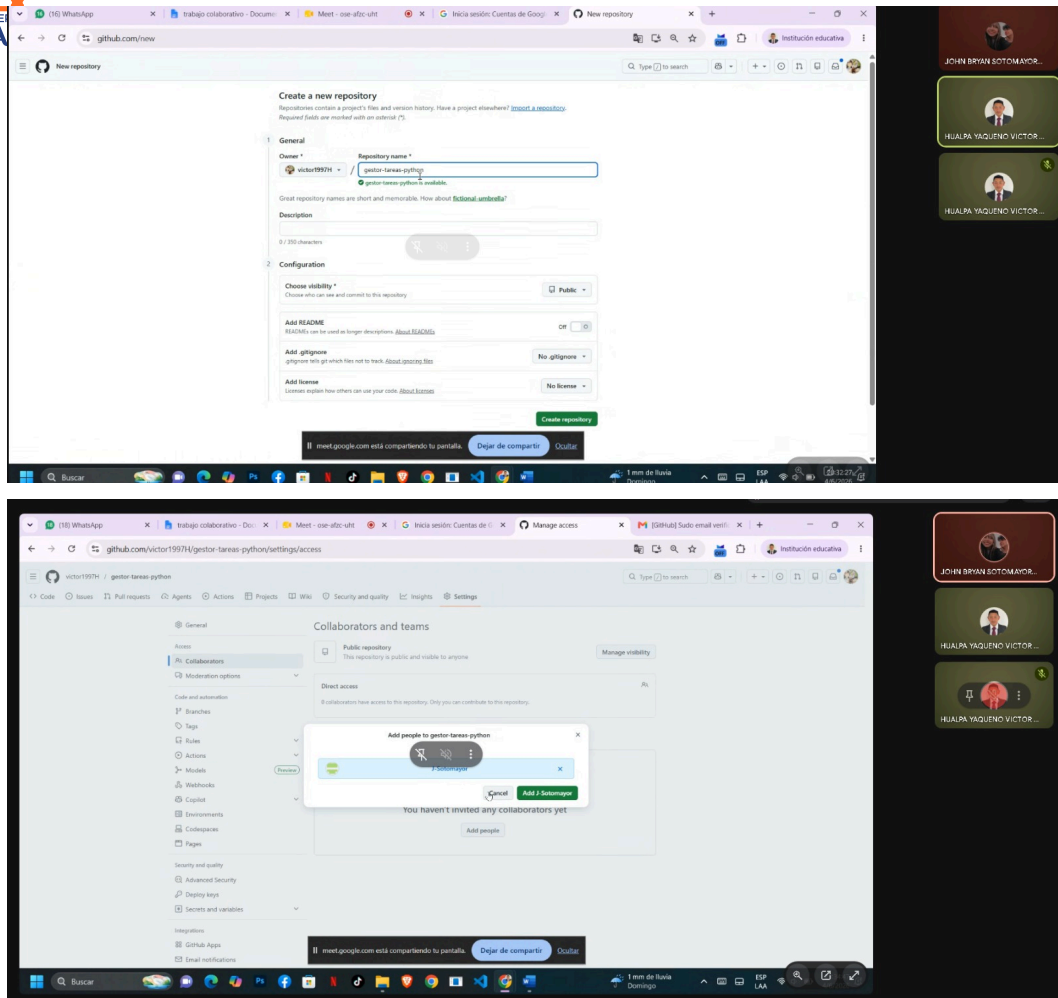
14. Despliegue Continuo (Continuous Deployment)

- **Definición:** Nivel máximo de automatización donde todo código que pasa las pruebas se publica en producción automáticamente.
- **Utilidad:** Elimina por completo la intervención humana en los lanzamientos, agilizando la entrega de valor a los clientes.
- **Ejemplo práctico:** Corrige un error ortográfico en tu web, haces un commit y en cuestión de minutos el usuario ya ve la corrección en vivo.
- **Fuente bibliográfica:** Red Hat. (s.f.). *¿Qué es CI/CD?*. Recuperado el 4 de junio de 2026, de <https://www.redhat.com/es/topics/devops/what-is-ci-cd>

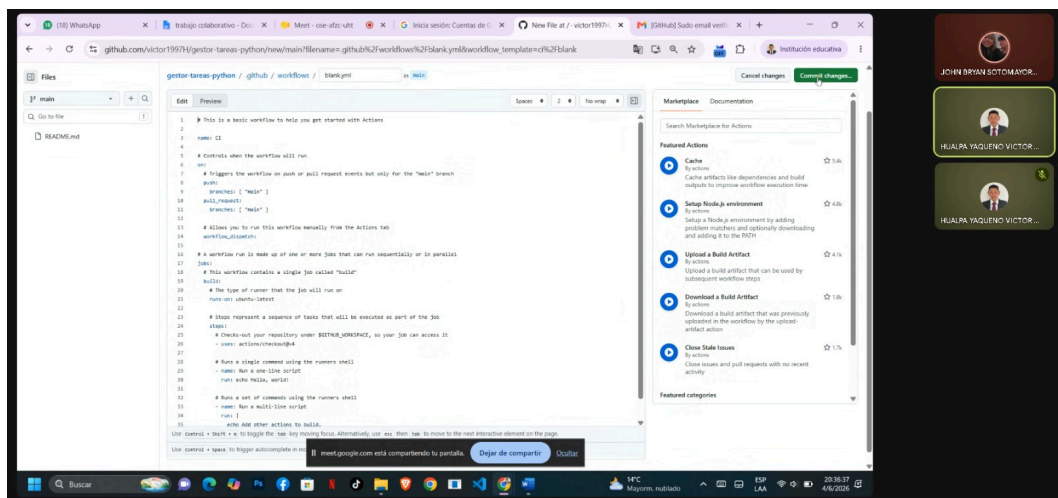
15. GitHub Actions

- **Definición:** Herramienta de GitHub que permite automatizar tareas y flujos de trabajo (como CI/CD) directamente en el repositorio.
- **Utilidad:** Ejecuta pruebas, alertas o despliegues automáticos sin tener que instalar aplicaciones externas.
- **Ejemplo práctico:** Configurar un archivo para que GitHub pruebe automáticamente el código cada vez que se crea un Pull Request.
- **Fuente bibliográfica:** GitHub Docs. (s.f.). *Comprender GitHub Actions*. Recuperado el 4 de junio de 2026, de <https://docs.github.com/es>

Fase 1: Configuración Inicial y GitHub Actions



- Figura 1: Configuración de permisos en la plataforma GitHub. El Estudiante: Victor Hualpa añade al Estudiante : John Sotomayor como colaborador autorizado en el repositorio remoto para permitir el desarrollo conjunto del gestor de tareas en Python.

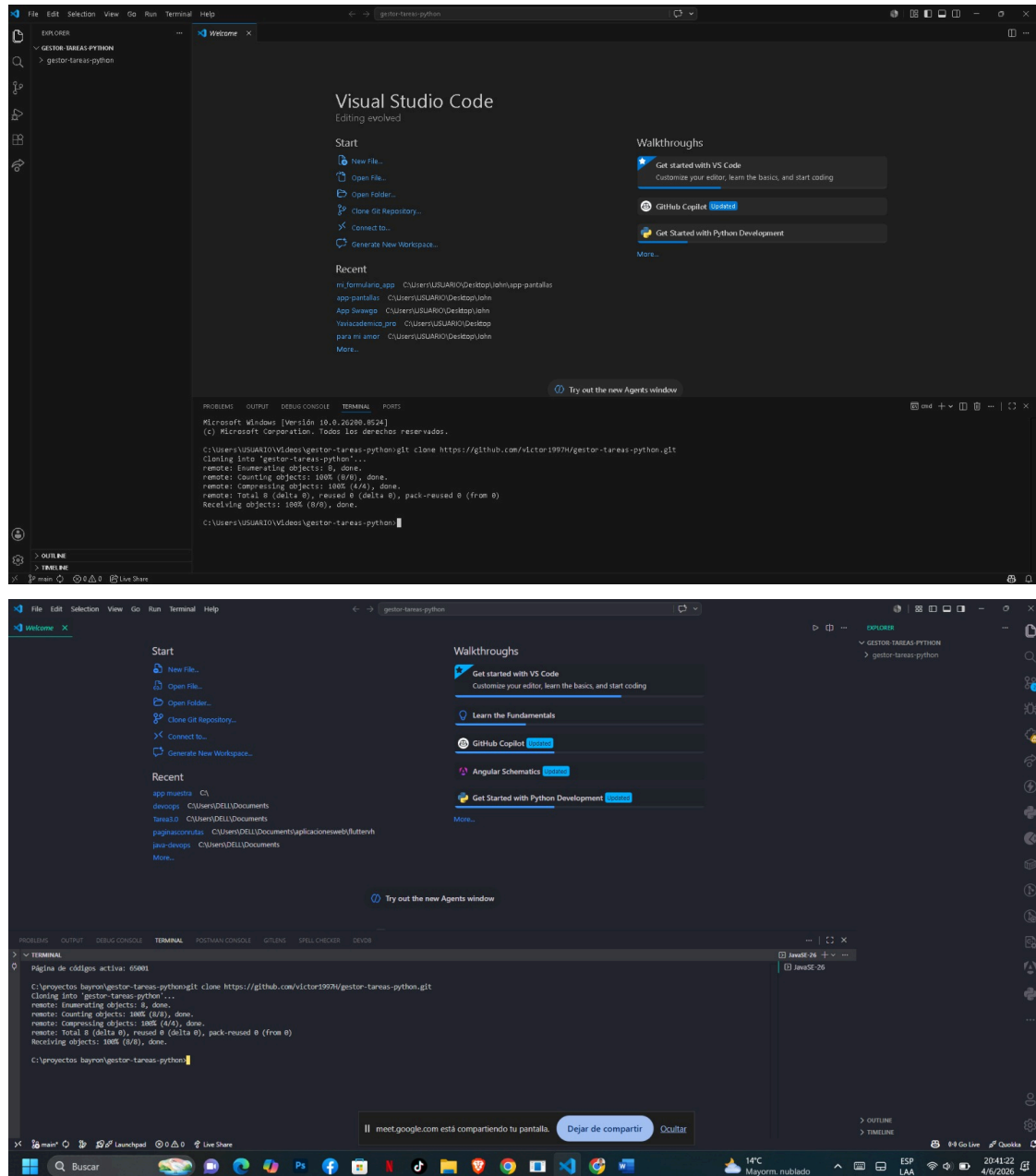


- Figura 2: Inicialización de procesos de Integración Continua. Se activa la herramienta GitHub Actions mediante la incorporación de un *Simple Workflow*

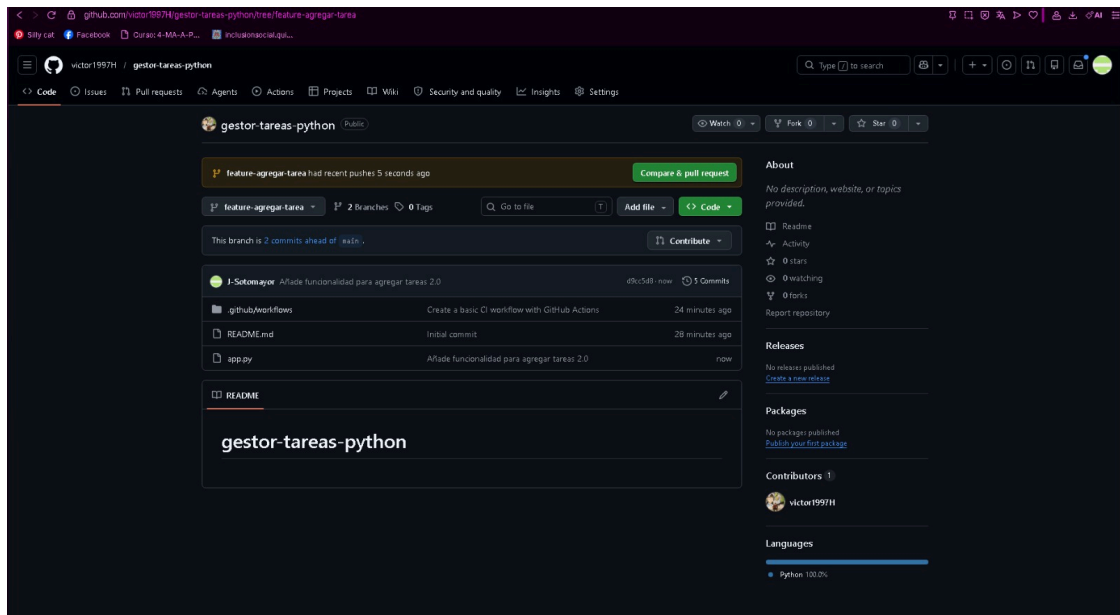
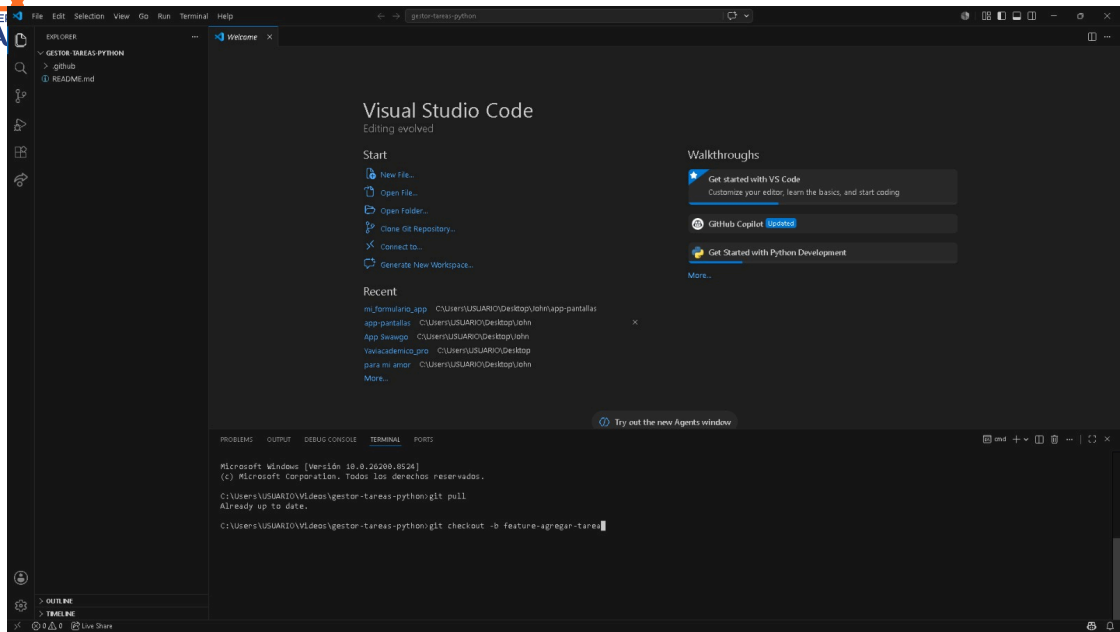


básico, evidenciando una ejecución exitosa señalada con el icono verde de validación.

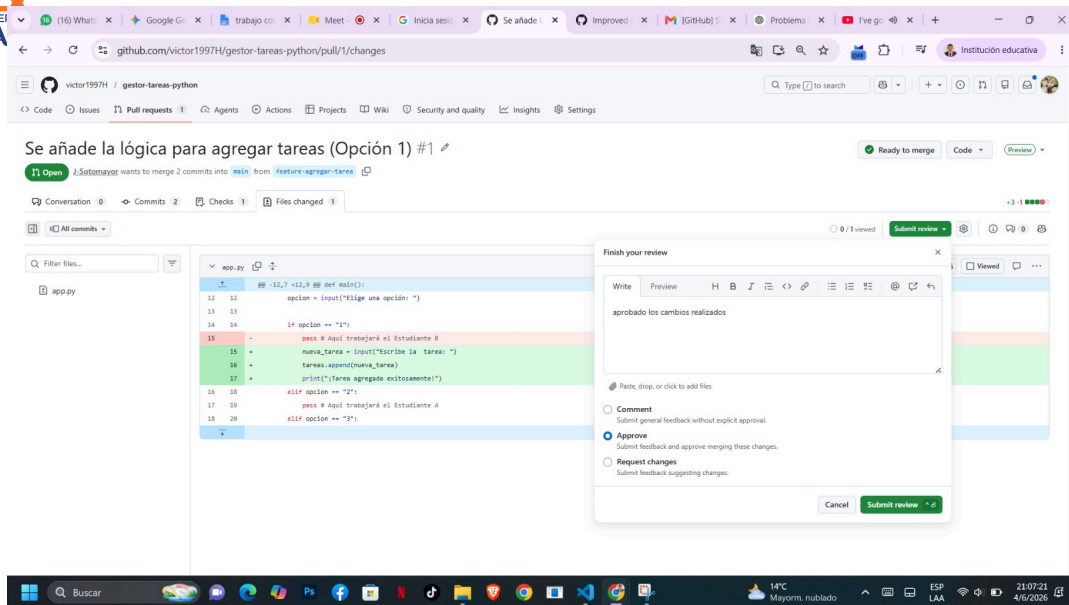
- **Fase 2: Estudiante :John Sotomayor Desarrolla / Estudiante :Victor Hualpa Revisa**



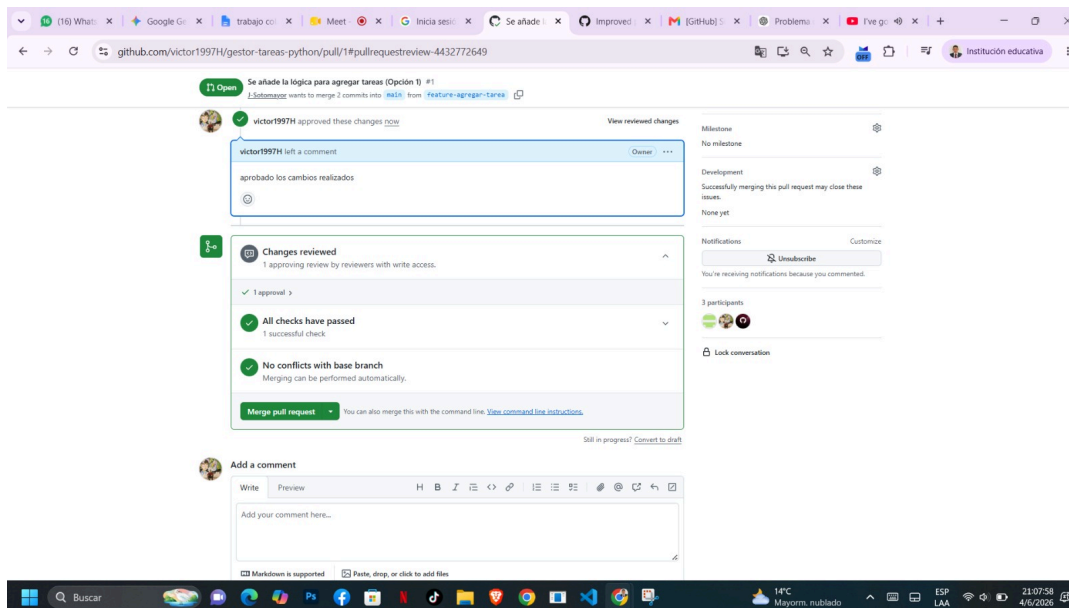
- **Figura 3: Trabajo en entorno local. El Estudiante :John Sotomayo crea la rama de desarrollo aislado feature-agregar-tarea mediante la consola de comandos y sube los cambios iniciales al servidor remoto.**

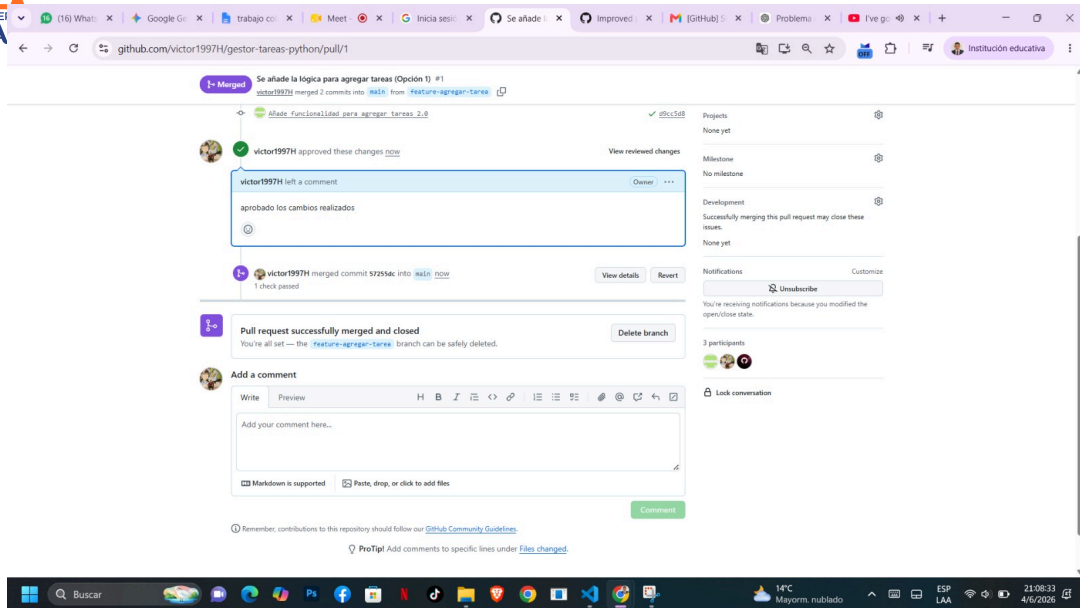


- Figura 4: Solicitud de revisión de código. El Estudiante :John Sotomayor genera formalmente un *Pull Request* en la interfaz web de GitHub para requerir la integración de la lógica que permite añadir tareas en el script `app.py`.

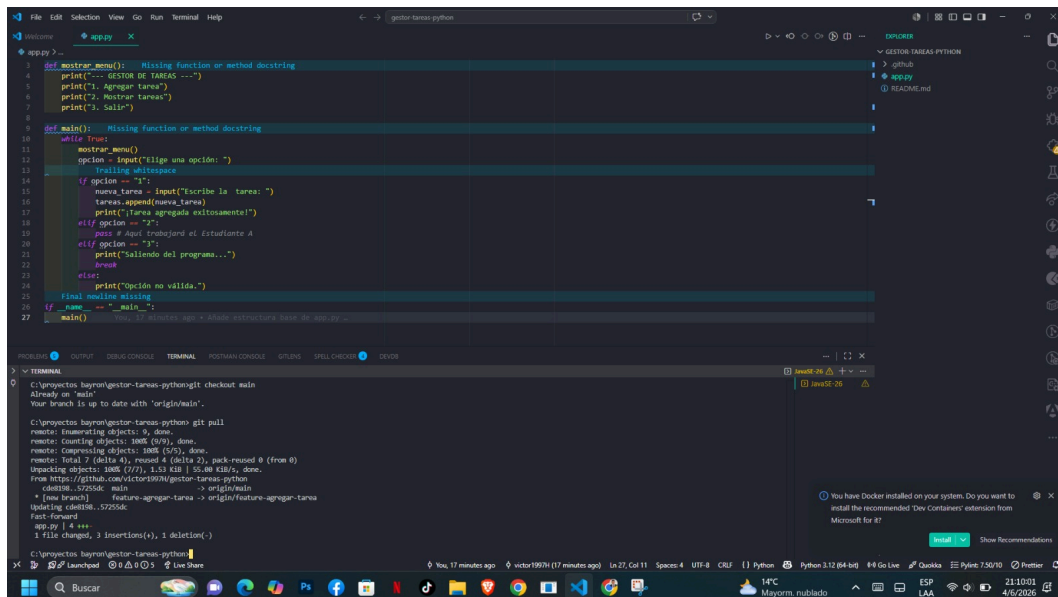


- Figura 5: Proceso de aseguramiento de calidad. El Estudiante: Victor Hualpa inspecciona las líneas de código añadidas en Python desde la pestaña *Files changed*, emitiendo un voto favorable y un comentario formal de aprobación (*Approve*).





- Figura 6: Cierre y fusión del ciclo de desarrollo. El Estudiante:Victor Hualpa consolida los cambios presionando el botón *Merge Pull Request*, unificando la función de inserción de tareas dentro de la rama principal main.
- **Fase 3: Cambio de Roles (Estudiante:Victor Hualpa Desarrolla / Estudiante:John Sotomayor Revisa)**



```

1 tareas = []
2
3 def mostrar_menu():
4     print("*** SUPER GESTOR DE TAREAS ***")
5     print("1.- Agregar tarea")
6     print("2.- Mostrar tareas")
7     print("Cerrando aplicación... Adiós")
8
9 def main():
10    while True:
11        mostrar_menu()
12        opcion = input("Elige una opción: ")
13        trailing_whitespace = opcion[-1]
14        if opcion == "1":
15            nueva_tarea = input("Escribe la tarea: ")
16            tareas.append(nueva_tarea)
17            print("Tarea agregada exitosamente!")
18            trailing_whitespace = opcion[-1]
19        elif opcion == "2":
20            print("----- TAREAS PENDIENTES -----")
21            for i, tarea in enumerate(tareas):
22                print(f"{i+1} - {tarea}")
23
24 if __name__ == "__main__":
25     main()

```

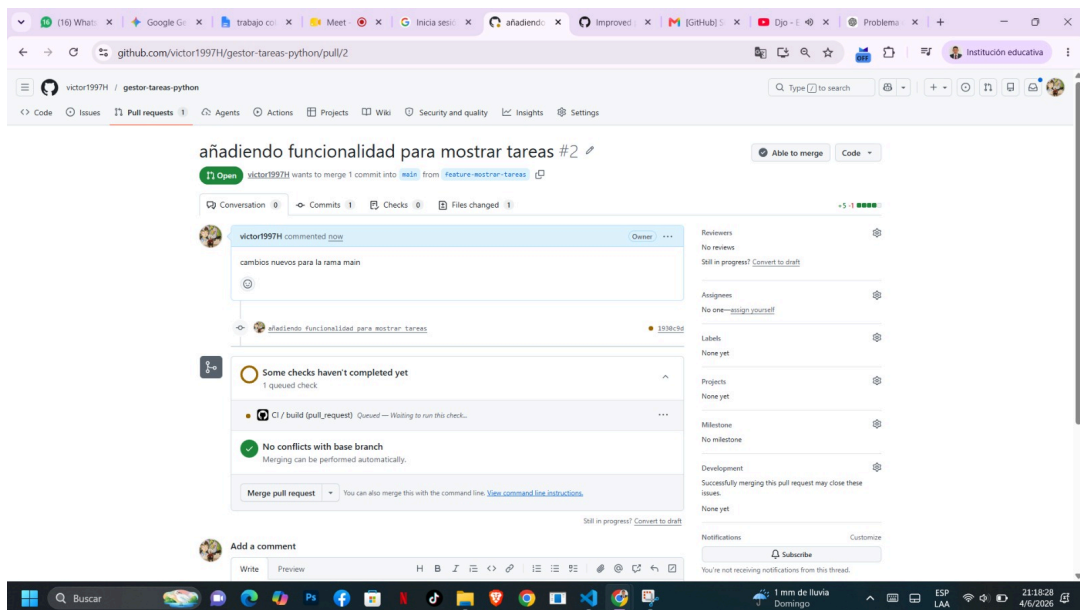
```

git push --set-upstream origin feature-mostrar-tareas
To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetUpstream' in 'git help config'.

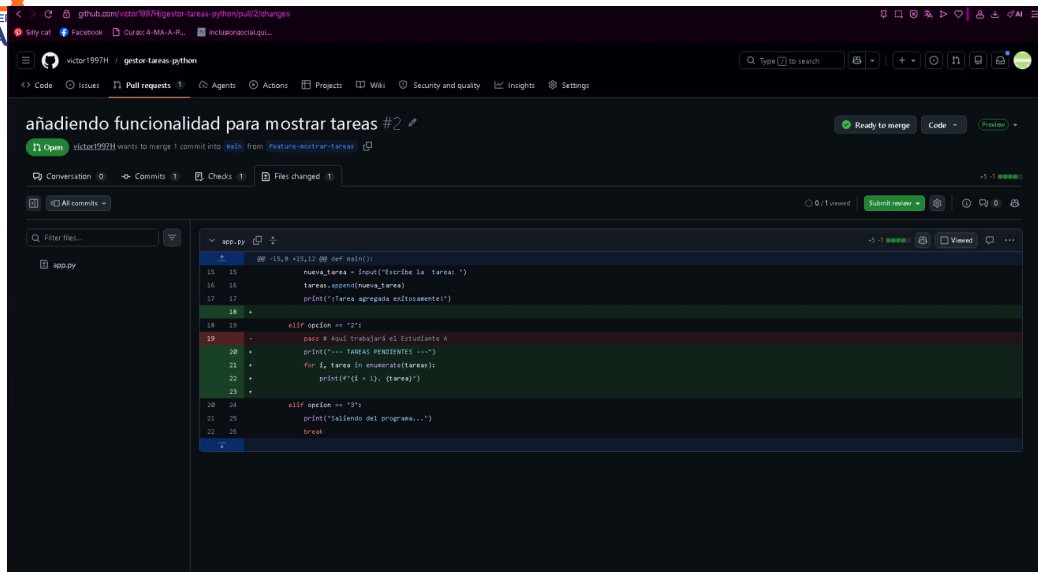
C:\proyectos\hayan\gestor-tareas-python>git push -u origin feature-mostrar-tareas
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 412 bytes | 412.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
remote: Create a pull request for 'feature-mostrar-tareas' on GitHub by visiting:
remote:   https://github.com/victor1997H/gestor-tareas-python/pull/new/feature-mostrar-tareas
remote:
[feature-mostrar-tareas] feature-mostrar-tareas -> feature-mostrar-tareas
branch 'feature-mostrar-tareas' set up to track 'origin/feature-mostrar-tareas'.
C:\proyectos\hayan\gestor-tareas-python>git push

```

- Figura 7: Inversión de roles en consola. El Estudiante:Victor Hualpa actualiza su espacio de trabajo, edita la sección del menú de visualización en el archivo app.py y publica la rama feature-mostrar-tareas.

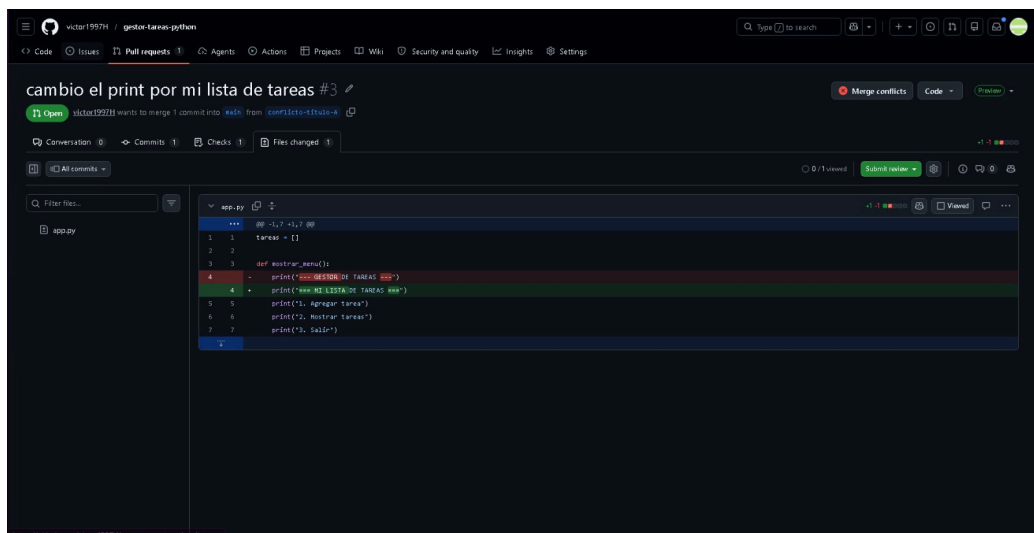


- Figura 8: Publicación de solicitud del Estudiante:Victor Hualpa. Se documenta la apertura de un nuevo *Pull Request* encaminado a incorporar la función que despliega la lista estructurada de tareas pendientes por pantalla.

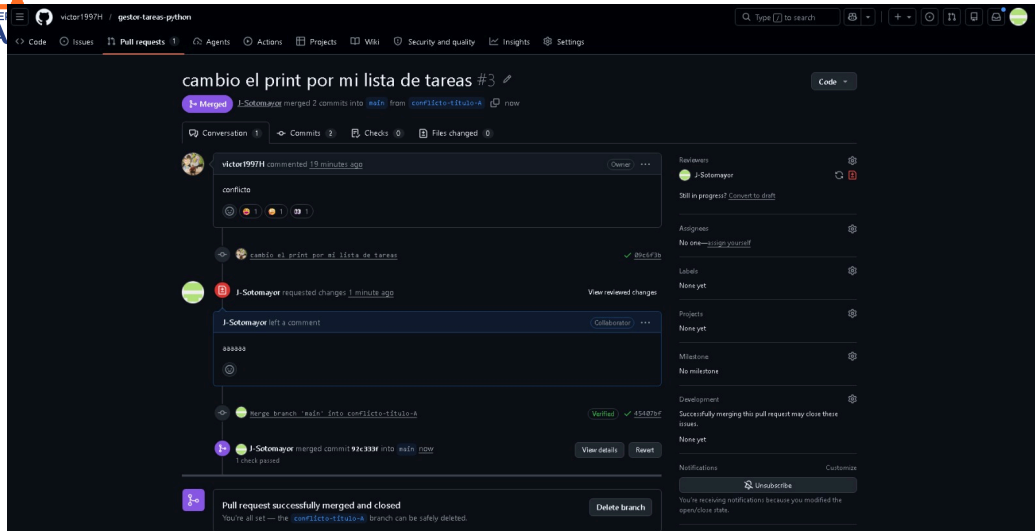


- Figura 9: Aprobación y fusión por el Estudiante : John Sotomayor. El Estudiante :John Sotomayor examina las líneas modificadas de la opción "2" del menú, valida su correcto funcionamiento y autoriza el *Merge* definitivo en la web.
- **Fase 4: Generación y Resolución de Conflictos**

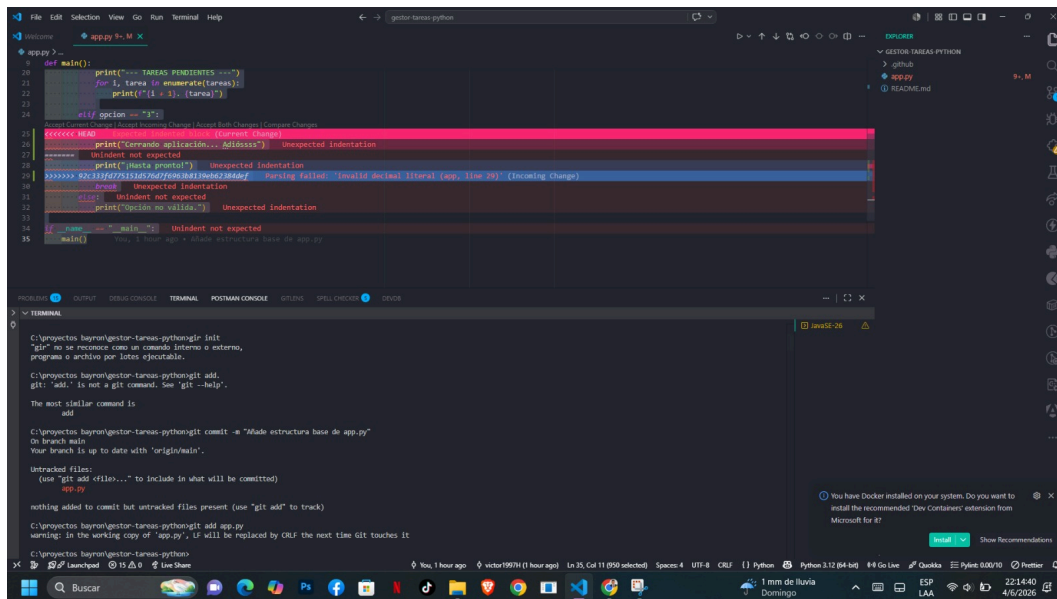
Ronda de Conflicto 1 (Estudiante : John Sotomayor resuelve)



- Figura 10: Bloqueo de integración por colisión de datos. GitHub detecta que ambos estudiantes modificaron la línea 4 de `app.py` con títulos incompatibles, deshabilitando la fusión automática del Pull Request del Estudiante : John Sotomayor.



- Figura 11: Resolución manual de conflicto de títulos. El Estudiante :John Sotomayo ingresa a la herramienta web de edición, remueve los delimitadores sintácticos de Git, selecciona el encabezado definitivo para el menú del programa y confirma el *Commit merge*.
- **Ronda de Conflicto 2 (Estudiante:Victor Hualpa resuelve)**



- Figura 12: Segundo evento de colisión de código. Se genera un conflicto cruzado en la opción "3" del gestor de tareas tras haberse modificado las cadenas de texto del comando `print` de despedida de forma simultánea.


```

def main():
    tareas = []
    while True:
        mostrar_menu()
        opcion = input("Elige una opción ")
        if opcion == "1":
            nueva_tarea = input("Escribe la tarea: ")
            tareas.append(nueva_tarea)
            print("Tarea agregada exitosamente!")
        elif opcion == "2":
            print("Tareas pendientes:")
            for i, tarea in enumerate(tareas):
                print(f"{i + 1}. {tarea}")
        elif opcion == "3":
            print("¡Manda pronto!")
            break
        else:
            print("Opción no válida.")
    if __name__ == "__main__":
        main()

```

```

(victor1997H) C:\projects\gestor-tareas-python> git add .
no changes added to commit (use "git add" and/or "git commit -a")
(victor1997H) C:\projects\gestor-tareas-python> git commit -a
[conflicto-salida-A Resolved] Resolved conflict in app.py
(victor1997H) C:\projects\gestor-tareas-python> git push
Counting objects: 100% (1/1), done.
Writing objects: 100% (1/1), 233 bytes | 233.00 KiB/s, done.
Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/victor1997H/gestor-tareas-python.git
  2xMach...>Mach... conflict-salida-A -> conflict-salida-A

```

- Figura 13: Cierre técnico del laboratorio. El Estudiante :Victor Hualpa corrige manualmente el conflicto borrando las marcas del archivo app . py, unifica la despedida de la aplicación y concreta exitosamente la integración final de todo el sistema colaborativo.
- **Estado Final del Proyecto**

```

1  tareas = []
2
3  def mostrar_menu():
4      print("=== SUPER GESTION DE TAREAS ===")
5      print("1. Agregar tarea")
6      print("2. Mostrar tareas")
7      print("3. Cerrar aplicación... Adios")
8
9  def main():
10     while True:
11         mostrar_menu()
12         opcion = input("Elige una opción ")
13
14         if opcion == "1":
15             nueva_tarea = input("Escribe la tarea: ")
16             tareas.append(nueva_tarea)
17             print("Tarea agregada exitosamente!")
18
19         elif opcion == "2":
20             print("Tareas pendientes:")
21             for i, tarea in enumerate(tareas):
22                 print(f"{i + 1}. {tarea}")
23
24         elif opcion == "3":
25             print("¡Manda pronto!")
26             break
27         else:
28             print("Opción no válida.")
29
30     if __name__ == "__main__":
31         main()

```

- Figura 14: Estado maestro de la aplicación en GitHub. Vista final limpia del archivo app . py en la rama principal, integrando armónicamente las funciones y títulos desarrollados colaborativamente por ambos estudiantes.

Url al repositorio del Github:

<https://github.com/victor1997H/gestor-tareas-python.git>

Conclusiones Individuales

Conclusiones del Estudiante Victor Hualpa: La realización de esta práctica práctica enfocada en el control de versiones me ha permitido comprender el valor de utilizar Git en entornos de desarrollo real, superando la antigua idea de trabajar sobre archivos estáticos compartidos de manera informal. El proceso de revisar los cambios del Estudiante John Sotomayor a través de la pestaña de inspección visual de GitHub me demostró la importancia de establecer una cultura de revisión de código previa a la integración, lo cual previene que errores lógicos ingresen a la producción del software. Asimismo, experimentar la aparición y resolución de conflictos me enseñó que Git no destruye información cuando hay colisiones de código, sino que actúa como un sistema de seguridad que obliga a los desarrolladores a dialogar y decidir técnicamente la mejor solución. Finalmente, la integración de GitHub Actions como herramienta nativa me abrió los ojos hacia la automatización, comprendiendo que los sistemas modernos son capaces de validar la integridad de una aplicación en Python sin necesidad de intervención manual constante, optimizando los tiempos del equipo.

Conclusiones del Estudiante John Sotomayor: Mi experiencia a lo largo de este laboratorio colaborativo fue sumamente enriquecedora, especialmente al enfrentar los retos técnicos que se presentaron desde la consola de comandos local. Al principio, lidiar con la falta de vinculación de las ramas locales y el servidor remoto me obligó a asimilar conceptos abstractos como el *upstream* y la sincronización de ramas secundarias individuales. El desarrollo estructurado de las funciones en Python me demostró cómo el uso correcto de las ramas permite a dos personas fragmentar una aplicación lógica y avanzar simultáneamente sin entorpecer los archivos base del otro. El momento más crítico de la práctica fue resolver el conflicto en el menú de la aplicación, lo que me ayudó a perder el miedo a las alertas de Git y a entender la estructura interna de los marcadores que señalan las líneas en disputa. Concluyó que dominar estas herramientas de software no es solo una competencia técnica individual, sino un requisito indispensable para la comunicación efectiva y el éxito operativo de cualquier proyecto tecnológico moderno.

BIBLIOGRAFÍA

Amazon Web Services. (s.f.). *¿Qué es la entrega continua?*. Recuperado el 4 de junio de 2026, de <https://aws.amazon.com/es/devops/continuous-delivery/>

Atlassian. (s.f.). *Tutoriales de Git*. Recuperado el 4 de junio de 2026, de <https://www.atlassian.com/es/git/tutorials>

Chacon, S., & Straub, B. (2015). *Pro Git: El libro oficial en español* (2.ª ed.). Apress. <https://git-scm.com/book/es/v2>

GitHub Docs. (s.f.). *Documentación oficial de GitHub*. Recuperado el 4 de junio de 2026, de <https://docs.github.com/es>



Red Hat (s.f.). *¿Qué es CI/CD? Conceptos clave de la Integración y Entrega Continuas.*

Recuperado el 4 de junio de 2026, de <https://www.redhat.com/es/topics/devops/what-is-ci-cd>